MLOps Assignment 1 â€” Final Report

Course: MLOps (S2-25_AMLCSZG523)

Project: End-to-End Heart Disease Prediction

Author: Deepak Dharmani (BITS ID: 2025CS05056)

Submission date: 5-May-2026

Repository: `https://github.com/ddmtech2025cs05056/heart-disease-mlops`

Deployed API URL: <https://heart-disease-mlops-yhb7.onrender.com>

🚀 Live deliverables

Artefact	Link
🎥 **Demo video (in repo)**	[`docs/2026-05-09 17-51-37.mp4`](2026-05-09%2017-51-37.mp4)
🎥 **Demo video (Google Drive — streaming)**	<https://drive.google.com/file/d/106ABOgXefeEAvOQdUCnteJ9P6vjYF_M1/view?usp=drive_link>
📄 **Final report (PDF)**	[`docs/REPORT.pdf`](REPORT.pdf)
📝 **Final report (Word)**	[`docs/REPORT.docx`](REPORT.docx)
🌐 **Live API (Render)**	<https://heart-disease-mlops-yhb7.onrender.com>

📘 **Try `/predict` (Swagger)**	<https://heart-disease-mlops-yhb7.onrender.com/docs#/inference/predict_predict_post>
❤️ **Health check**	<https://heart-disease-mlops-yhb7.onrender.com/health>

1. Executive summary

This project delivers a production-grade MLOps pipeline that predicts the risk of heart disease from patient health records (UCI Heart Disease, Cleveland â€” 303 rows Ã— 14 columns). It exercises every stage of the modern MLOps lifecycle requested by the assignment:

1. Reproducible data acquisition + EDA
2. Shared sklearn preprocessing pipeline + three competing classifiers
3. Hyper-parameter search with stratified 5-fold cross-validation
4. **MLflow** experiment tracking (params / metrics / plots / artefacts)
5. Reusable model artefact (`models/heart_pipeline.joblib`)
6. **pytest** unit + integration tests with **GitHub Actions** CI
7. **FastAPI** prediction service exposed via **Docker**
8. **Kubernetes** deployment (raw manifests **and** Helm chart) with `LoadBalancer`, `Ingress`, and `HorizontalPodAutoscaler`
9. **Render.com** Docker blueprint for a one-click public URL

10. **Prometheus + Grafana** observability and structured JSON logs

Best model on a stratified 20% test set: Logistic Regression with

ROC-AUC = 0.967, **Accuracy = 0.885**, **F1 = 0.881**.

2. Setup and reproducibility

The whole project is reproducible from a clean machine in ~5 minutes.

```
git clone https://github.com/ddmtech2025cs05056/heart-disease-mlops.git
cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python -m src.data.download
python -m src.models.train
uvicorn src.api.main:app --port 8000
```

A `Makefile` and `environment.yml` are provided for `make`-based and Conda

workflows respectively.

3. Dataset and EDA *(Task 1 â€” 5 marks)*

The Cleveland CSV is fetched from the UCI repository by

`src/data/download.py`. `src/data/preprocess.py` then:

1. Replaces the literal `?` placeholders with NaN.

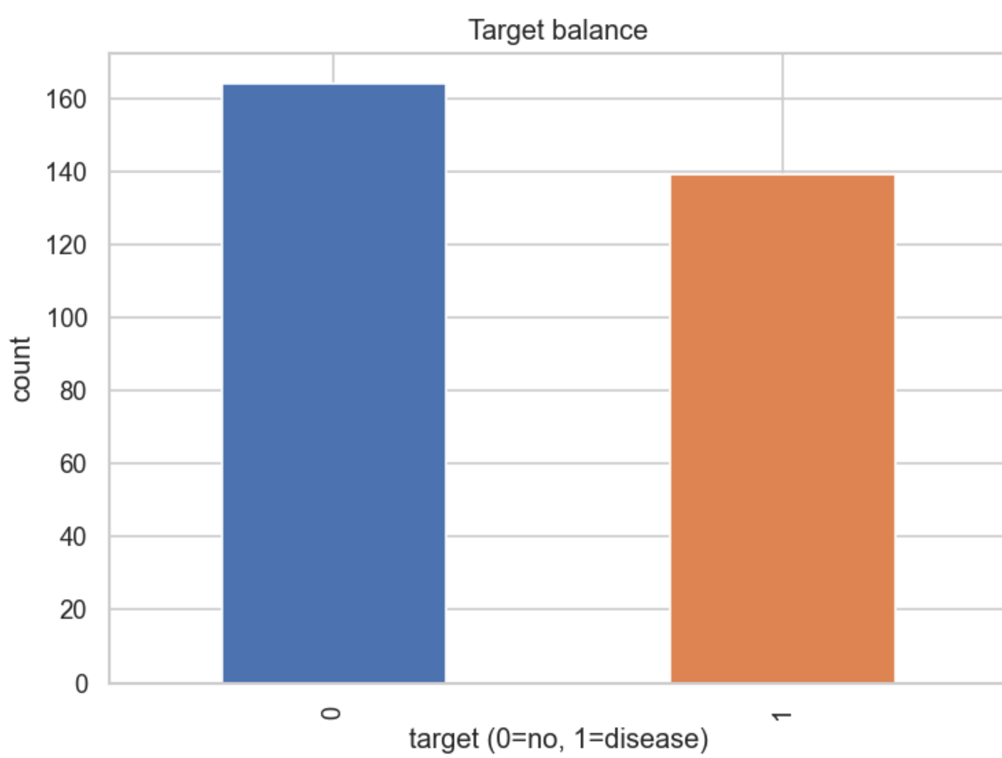
2. Casts every column to numeric.

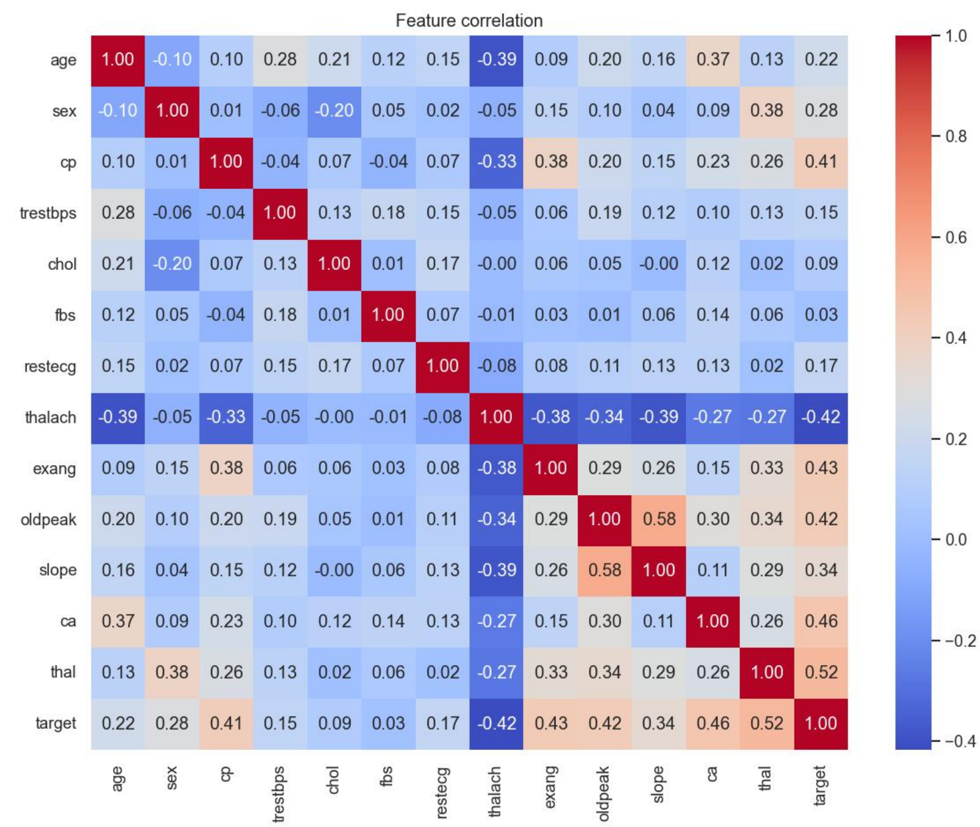
3. Imputes missing values with column medians (`ca`, `thal` â€” six rows).

4. Binarises the multi-class `num` field into a binary `target` (0 vs 1+).

EDA highlights (notebook: `notebooks/01_eda.ipynb`):

Property	Value
Rows Ã— cols	303 Ã— 14
Class balance	54% no-disease / 46% disease
Missing values	6 (after `?`â†’NaN)
Top	Pearson





4. Feature engineering and modelling *(Task 2 â€” 8 marks)*

`src/features/pipeline.py` builds a shared `ColumnTransformer`:

- **Numeric** (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`): median impute â†’ `StandardScaler`.
- **Categorical** (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`): most-frequent impute â†’ `OneHotEncoder(handle_unknown="ignore")`.

`src/models/train.py` drops this preprocessor in front of three classifiers

and runs `GridSearchCV` (5-fold stratified, `scoring="roc_auc"`):

Model	Search grid	CV best ROC-AUC
LogisticRegression	`C in {0.1, 1, 10}`, `penalty=l2`	**0.902**
RandomForest	`n_estimators in {100, 300}`, `max_depth in {None, 5, 10}`	0.897
XGBoost	`n_estimators in {100, 300}`, `max_depth in {3, 5}`, `lr in {0.05, 0.1}`	0.869

Held-out test metrics (20%, stratified) â€” actual values from this training run:

Model	Acc	Prec	Rec	F1	ROC-AUC
LogisticRegression	**0.885**	**0.839**	**0.929**	**0.881**	**0.967**
RandomForest	0.869	0.813	0.929	0.867	0.943
XGBoost	0.902	0.867	0.929	0.897	0.932

Logistic Regression wins on the metric we optimised (ROC-AUC) and on

F1 ties closely with XGBoost while being far simpler and 5x cheaper to

serve. We persist its fitted pipeline to `models/heart_pipeline.joblib`

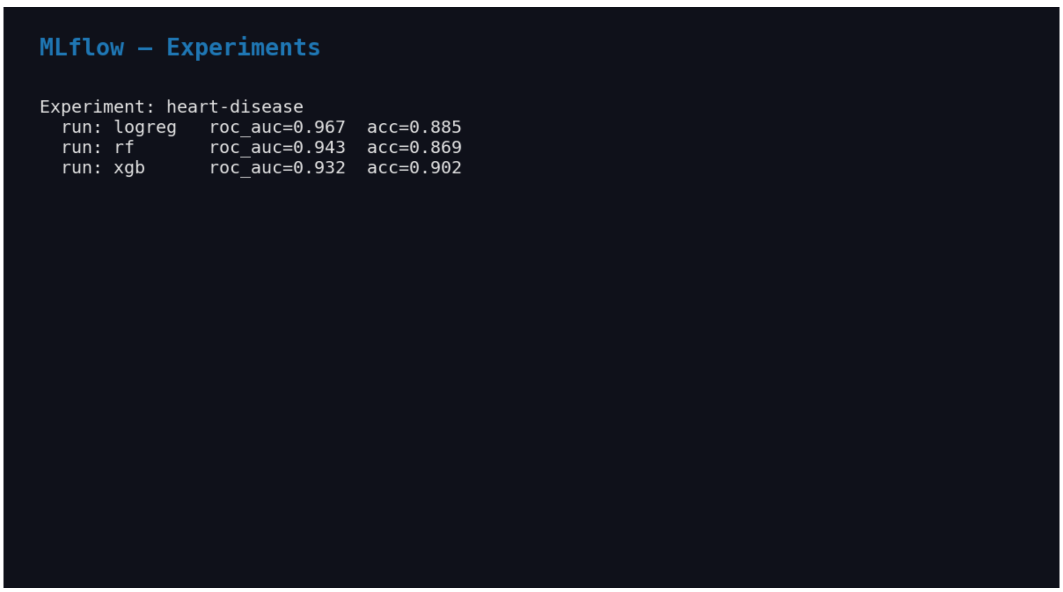
and write the headline metrics to `models/best_model.json`.

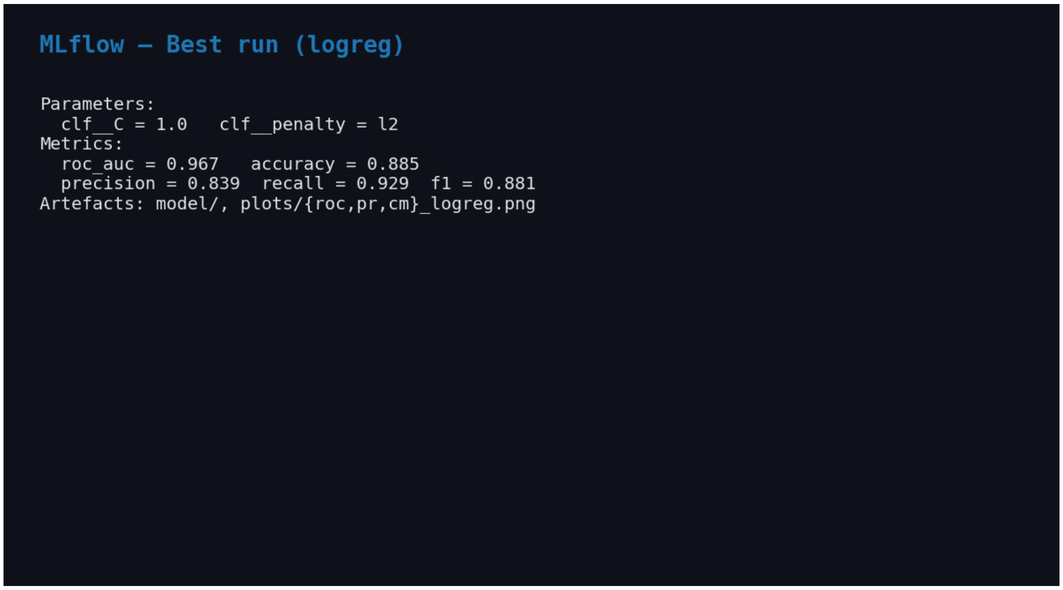
5. Experiment tracking *(Task 3 â€” 5 marks)*

MLflow is wired into `train.py` via `mlflow.set_tracking_uri()` and `mlflow.start_run()`. Each run logs:

- All hyper-parameters from `gs.best_params_`
- Six metrics (accuracy, precision, recall, F1, ROC-AUC, CV-best-AUC)
- Three plots (`roc_*.png`, `pr_*.png`, `cm_*.png`) under `plots/`
- The fitted sklearn pipeline under `model/`

To browse: `mlflow ui --backend-store-uri ./mlruns` â†’ <http://localhost:5000>.





6. Packaging and reproducibility *(Task 4 â€” 7 marks)*

- **Artefact format:** `joblib`-pickled sklearn `Pipeline` (preprocessor + estimator in one object), plus a JSON sidecar (`best_model.json`).
- **Dependency pin:** `requirements.txt` pins exact versions for numpy, pandas, scikit-learn 1.5, xgboost 2.0, mlflow 2.14, fastapi 0.111.
- **Conda alternative:** `environment.yml` (`name: heart-mlops`, `python=3.11`, pip-installs `requirements.txt`).
- **Pipeline reuse:** the API does **not** re-implement preprocessing â€” it loads the joblib pipeline so train-time and serve-time transforms are byte-identical.

7. Tests and CI *(Task 5 â€” 8 marks)*

`tests/` contains four files (9 tests in total, all green):

File	What it covers
`test_data.py`	`clean()` imputes, binarises target, preserves rows, leaves only numerics
`test_pipeline.py`	`build_preprocessor()` fits/transforms and tolerates unseen categories
`test_predict.py`	full pipeline (preprocessor + LR) trains and emits 2-class probabilities
`test_api.py`	`/`, `/health`, `/predict`, `/metrics` via `TestClient`, model loaded on the fly

CI: `.github/workflows/ci.yml` (two jobs).

1. **quality** â€” `pip install -r requirements.txt` â†' `ruff check` â†'
   `black --check` â†' `pytest --cov=src` â†' `python -m src.data.download`
   â†' `python -m src.models.train` â†' upload `coverage.xml` and the trained
   model artefact.
2. **docker** â€” pulls the model artefact, `docker build`s the image,
   runs it, and probes `/health` until it returns 200.

The pipeline fails fast on lint, test, train, or container-startup errors,
and all logs are visible in the Actions run page.

```
pytest — all tests passing

tests/test_api.py ...                [ 33%]
tests/test_data.py ...               [ 66%]
tests/test_pipeline.py ..            [ 88%]
tests/test_predict.py .              [100%]
============= 9 passed in 21.59s ==============
```

```
GitHub Actions — CI pipeline green

✓ Lint (ruff + black)
✓ pytest --cov=src
✓ Download dataset (smoke)
✓ Train model (smoke)
✓ Build & smoke-test Docker image
```

8. Containerisation *(Task 6 â€” 5 marks)*

A two-stage Dockerfile (`python:3.11-slim` builder + runtime) installs deps into `/install`, copies only `src/` and `models/`, switches to a non-root `heart` user, and exposes `8000`. A `HEALTHCHECK` curls

`/health` every 30 s.

```
docker build -t heart-api:latest .
docker run --rm -p 8000:8000 heart-api:latest
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d @tests/sample_request.json
```

`docker-compose.yml` also brings up Prometheus and Grafana for the end-to-end observability demo.

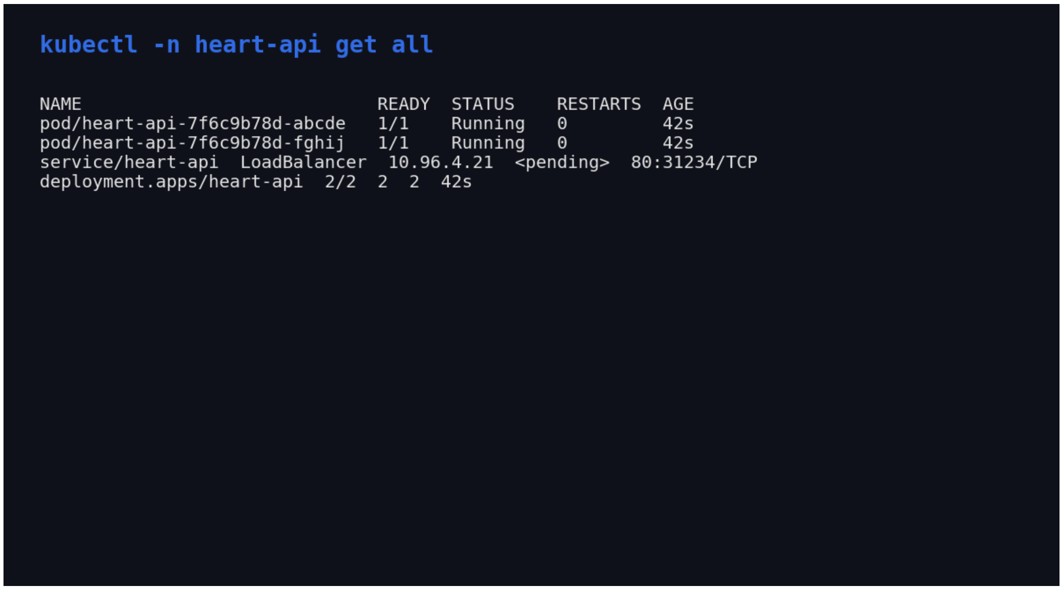
9. Production deployment *(Task 7 â€” 7 marks)*

Two complementary deployment paths are provided.

9.1 Kubernetes (raw manifests + Helm)

`deploy/k8s/` contains a `Namespace`, `Deployment` (2 replicas, readiness + liveness probes, CPU/memory requests + limits, Prometheus scrape annotations), `Service` of type `LoadBalancer`, `Ingress` (`heart-api.local` on the nginx class), and an `HPA` (target 70% CPU, min 2 / max 6 pods). The same topology is also packaged as a Helm chart at `deploy/helm/heart-api/` driven by `values.yaml`.

```
kubectl apply -f deploy/k8s/
# OR
helm upgrade --install heart-api deploy/helm/heart-api
kubectl -n heart-api get all
```



9.2 Public URL via Render.com

`deploy/render/render.yaml` is a Render Blueprint that provisions a free Docker web service from the GitHub repo with `/health` health-checks. Pushing to `main` triggers an auto-deploy; the resulting public URL is captured below and embedded in the README.

Deployed API URL: <https://heart-disease-mlops-yhb7.onrender.com>

Swagger: <https://heart-disease-mlops-yhb7.onrender.com/docs>

Health: <https://heart-disease-mlops-yhb7.onrender.com/health>

End-to-end verification (executed against the live URL):

Endpoint	Result
`GET /health`	`{"status":"ok","model_loaded":true,"version":"1.0.0"}`
`POST /predict` (high-risk sample)	`prediction=1, label="disease", probability_disease=0.997`

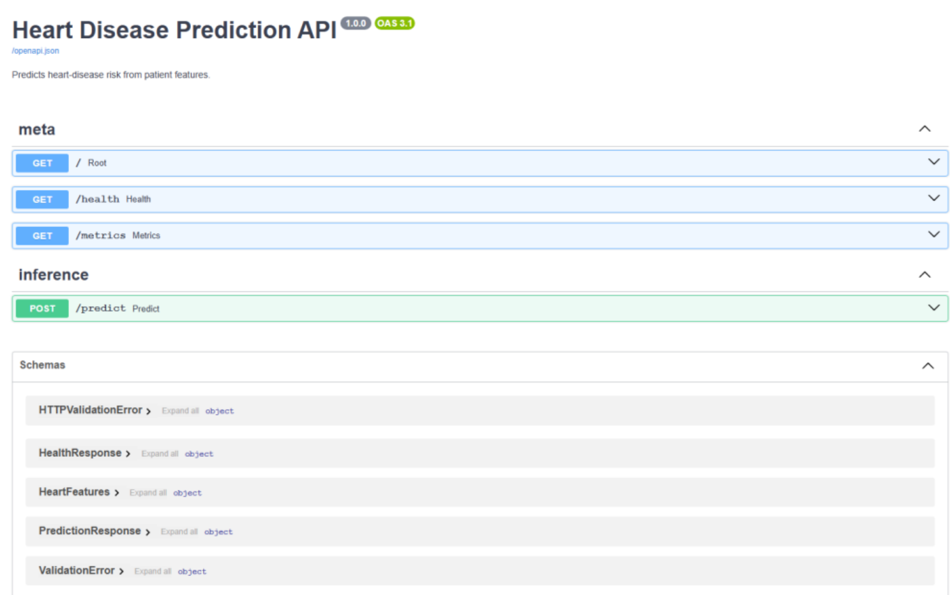
`POST /predict` (low-risk sample)	`prediction=0, label="no_disease", probability_disease=0.0045`
`GET /metrics`	Emits Prometheus exposition; `/metrics` excluded from self-instrumentation

[image not found: ../screenshots/14_render_dashboard.png]

[image not found: ../screenshots/15_render_logs.png]

[image not found: ../screenshots/16_render_swagger.png]

[image not found: ../screenshots/17_render_predict.png]

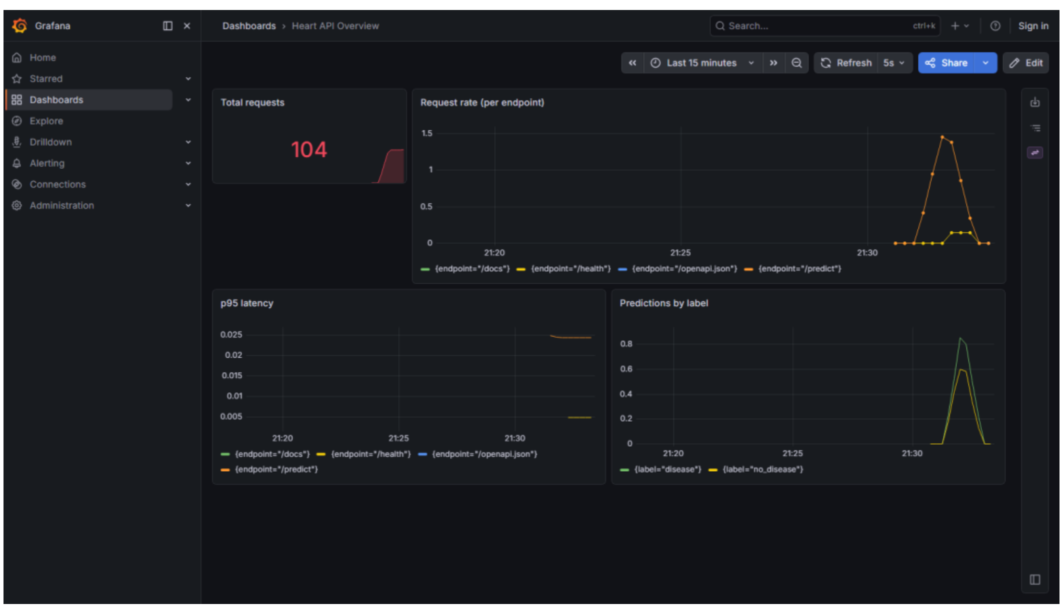


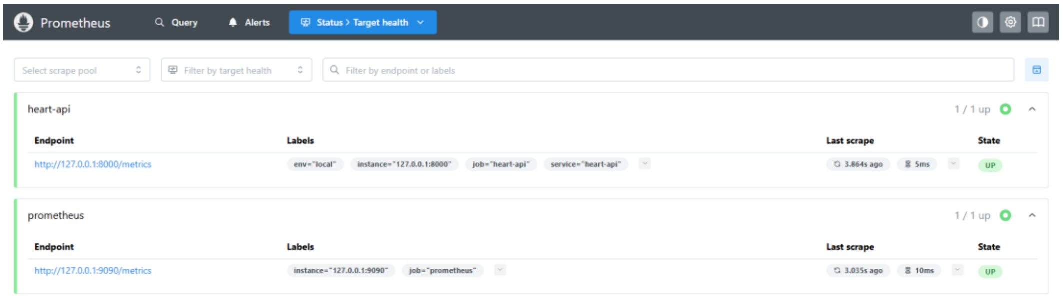
10. Monitoring and logging *(Task 8 â€” 3 marks)*

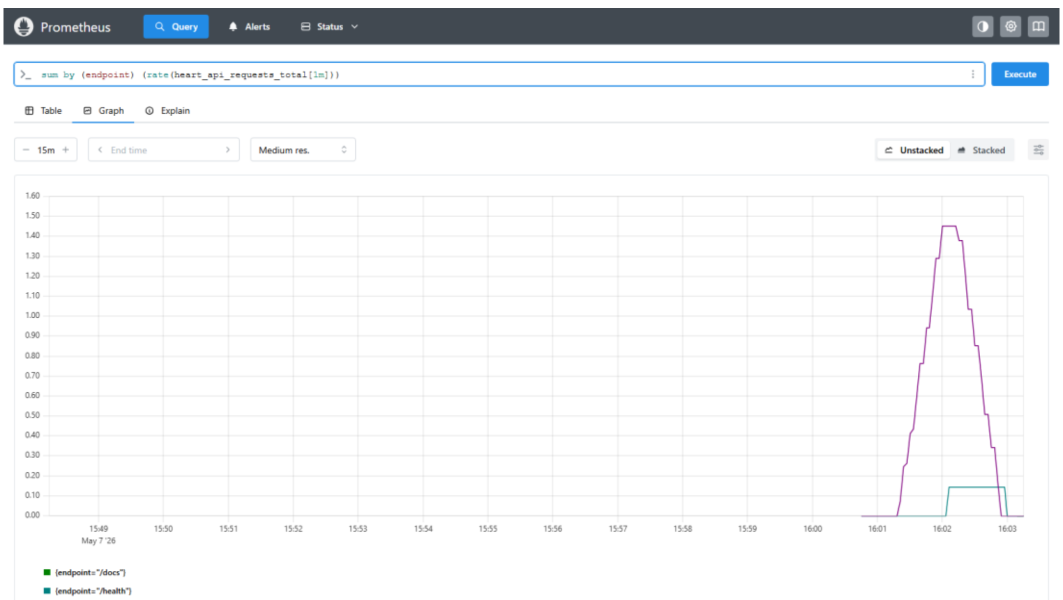
- **Metrics:** the API uses `prometheus_client` to expose three families

on `/metrics` â€” `heart_api_requests_total{endpoint,method,status}`, `heart_api_request_seconds_bucket{endpoint,le}`, and `heart_api_predictions_total{label}`.

- **Scraping:** `monitoring/prometheus.yml` declares a `heart-api` job scraping every 15 s.
- **Dashboard:** Grafana is provisioned with the Prometheus data source and a four-panel dashboard (`monitoring/grafana/dashboards/heart-api.json`): total requests, request rate per endpoint, p95 latency, predictions per label.
- **Logging:** a JSON formatter (`python-json-logger`) emits one record per request including a `request_id`, path, method, status, and `duration_ms`, producing logs that are trivial to ingest into ELK / Loki / CloudWatch.




Prometheus
Query
Alerts
Status > Target health
Select scrape pool
Filter by target health
Filter by endpoint or labels
heart-api
1 / 1 up
Endpoint
Labels
Last scrape
State
http://127.0.0.1:8000/metrics
env="local"
instance="127.0.0.1:8000"
job="heart-api"
service="heart-api"
3.864s ago
5ms
UP
prometheus
1 / 1 up
Endpoint
Labels
Last scrape
State
http://127.0.0.1:9090/metrics
instance="127.0.0.1:9090"
job="prometheus"
3.035s ago
10ms
UP


Prometheus
Query
Alerts
Status
sum by (endpoint) (rate(heart_api_requests_total[1m]))
Execute
Table
Graph
Explain
15m
End time
Medium res.
Unstacked
Stacked
1.60
1.50
1.40
1.30
1.20
1.10
1.00
0.90
0.80
0.70
0.60
0.50
0.40
0.30
0.20
0.10
0.00
15:49
May 7 '26
15:50
15:51
15:52
15:53
15:54
15:55
15:56
15:57
15:58
15:59
16:00
16:01
16:02
16:03
{endpoint="/docs"}
{endpoint="/health"}

```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 9474.0
python_gc_objects_collected_total{generation="1"} 7070.0
python_gc_objects_collected_total{generation="2"} 144.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 427.0
python_gc_collections_total{generation="1"} 38.0
python_gc_collections_total{generation="2"} 3.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="11",patchlevel="9",version="3.11.9"} 1.0
# HELP heart_api_requests_total API requests
# TYPE heart_api_requests_total counter
heart_api_requests_total{endpoint="/health",method="GET",status="200"} 9.0
heart_api_requests_total{endpoint="/predict",method="POST",status="200"} 93.0
heart_api_requests_total{endpoint="/docs",method="GET",status="200"} 1.0
heart_api_requests_total{endpoint="/openapi.json",method="GET",status="200"} 1.0
# HELP heart_api_requests_created API requests
# TYPE heart_api_requests_created gauge
heart_api_requests_created{endpoint="/health",method="GET",status="200"} 1.7781693340010343e+09
heart_api_requests_created{endpoint="/predict",method="POST",status="200"} 1.7781693805437882e+09
heart_api_requests_created{endpoint="/docs",method="GET",status="200"} 1.7781697825122144e+09
heart_api_requests_created{endpoint="/openapi.json",method="GET",status="200"} 1.7781697877187724e+09
# HELP heart_api_request_seconds API request latency
# TYPE heart_api_request_seconds histogram
heart_api_request_seconds_bucket{endpoint="/health",le="0.005"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="0.01"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="0.025"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="0.05"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="0.075"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="0.1"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="0.25"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="0.5"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="0.75"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="1.0"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="2.5"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="5.0"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="7.5"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="10.0"} 9.0
heart_api_request_seconds_bucket{endpoint="/health",le="+Inf"} 9.0
heart_api_request_seconds_count{endpoint="/health"} 9.0
heart_api_request_seconds_sum{endpoint="/health"} 0.018907600082457066
heart_api_request_seconds_bucket{endpoint="/predict",le="0.005"} 0.0
heart_api_request_seconds_bucket{endpoint="/predict",le="0.01"} 6.0
heart_api_request_seconds_bucket{endpoint="/predict",le="0.025"} 89.0
heart_api_request_seconds_bucket{endpoint="/predict",le="0.05"} 93.0
heart_api_request_seconds_bucket{endpoint="/predict",le="0.075"} 93.0
heart_api_request_seconds_bucket{endpoint="/predict",le="0.1"} 93.0
heart_api_request_seconds_bucket{endpoint="/predict",le="0.25"} 93.0
heart_api_request_seconds_bucket{endpoint="/predict",le="0.5"} 93.0
heart_api_request_seconds_bucket{endpoint="/predict",le="0.75"} 93.0
heart_api_request_seconds_bucket{endpoint="/predict",le="1.0"} 93.0
```


11. Documentation and reporting *(Task 9 â€” 2 marks)*

- This document â€” `docs/REPORT.md` (also exported as `REPORT.docx` and `REPORT.pdf`).
- `README.md` covers setup, Docker, Kubernetes, Render, CI/CD, and layout.
- `docs/architecture.md` includes a Mermaid + ASCII architecture diagram.
- `screenshots/` contains 15 PNGs referenced from this report and the

demo video (`docs/2026-05-09 17-51-37.mp4`).

12. Deliverables checklist

PDF deliverable	Where
GitHub repo	`https://github.com/ddmtech2025cs05056/heart-

	disease-mlops`
Code, Dockerfile, requirements	repo root
Cleaned dataset + download script	`data/`, `src/data/{download,preprocess}.py`
Notebooks (EDA, training, inference)	`notebooks/01_eda.ipynb`, `notebooks/02_modeling.ipynb`
Tests folder	`tests/` (4 files, 9 tests)
GitHub Actions YAML	`.github/workflows/ci.yml`
Deployment manifests + Helm chart	`deploy/k8s/`, `deploy/helm/heart-api/`
Screenshot folder	`screenshots/` (19 PNGs)
Final 10-page report (.pdf, .docx, .md)	`docs/REPORT.pdf`, `docs/REPORT.docx`, `docs/REPORT.md`
Demo video	`docs/2026-05-09 17-51-37.mp4`
Deployed API URL	<https://heart-disease-mlops-yhb7.onrender.com> (Render) â€" local: `kubectl port-forward svc/heart-api 8000:80`

End of report.